

SUBQUERIES AND NESTED QUERIES

1. Create table and insert values

Firstly, Created the table Employees and Departments as we are going to see about Subqueries and Nested queries..

```
SQLQuery3.sql... \Krishy (53))* -p X
3      use Employees
4
5      -- Employee table
6      CREATE TABLE employees (
7          employee_id INT PRIMARY KEY,
8          first_name VARCHAR(50),
9          last_name VARCHAR(50),
10         email VARCHAR(100),
11         salary DECIMAL(10, 2),
12         department_id INT
13     );
14
15     -- Inserting data into employees table
16     INSERT INTO employees (employee_id, first_name, last_name, email, salary, department_id) VALUES
17     (101, 'John', 'Doe', 'john.d@example.com', 60000.00, 10),
18     (102, 'Jane', 'Smith', 'jane.s@example.com', 75000.00, 20),
19     (103, 'Peter', 'Jones', 'peter.j@example.com', 55000.00, 10),
20     (104, 'Mary', 'Johnson', 'mary.j@example.com', 90000.00, 30),
21     (105, 'Chris', 'Lee', 'chris.l@example.com', 62000.00, 20);
22
23     -- department table
24
25     CREATE TABLE departments (
26         department_id INT PRIMARY KEY,
27         department_name VARCHAR(50)
28     );
29
30     -- Inserting data into department table
31     INSERT INTO departments (department_id, department_name) VALUES
32     (10, 'IT'),
33     (20, 'Sales'),
34     (30, 'Finance');
35
36     select * from employees
37     select * from departments
```

100 % No issues found

Messages

(3 rows affected)

Completion time: 2025-08-17T20:18:36.3029673+05:30

```
SQLQuery3.sql... \Krishy (53))* -p X
22
23     -- department table
24     CREATE TABLE departments (
25         department_id INT PRIMARY KEY,
26         department_name VARCHAR(50)
27     );
28
29     -- Inserting data into department table
30     INSERT INTO departments (department_id, department_name) VALUES
31     (10, 'IT'),
32     (20, 'Sales'),
33     (30, 'Finance');
34
35     select * from employees
36     select * from departments
```

100 % No issues found

Results Messages

	employee_id	first_name	last_name	email	salary	department_id
1	101	John	Doe	john.d@example.com	60000.00	10
2	102	Jane	Smith	jane.s@example.com	75000.00	20
3	103	Peter	Jones	peter.j@example.com	55000.00	10
4	104	Mary	Johnson	mary.j@example.com	90000.00	30
5	105	Chris	Lee	chris.l@example.com	62000.00	20

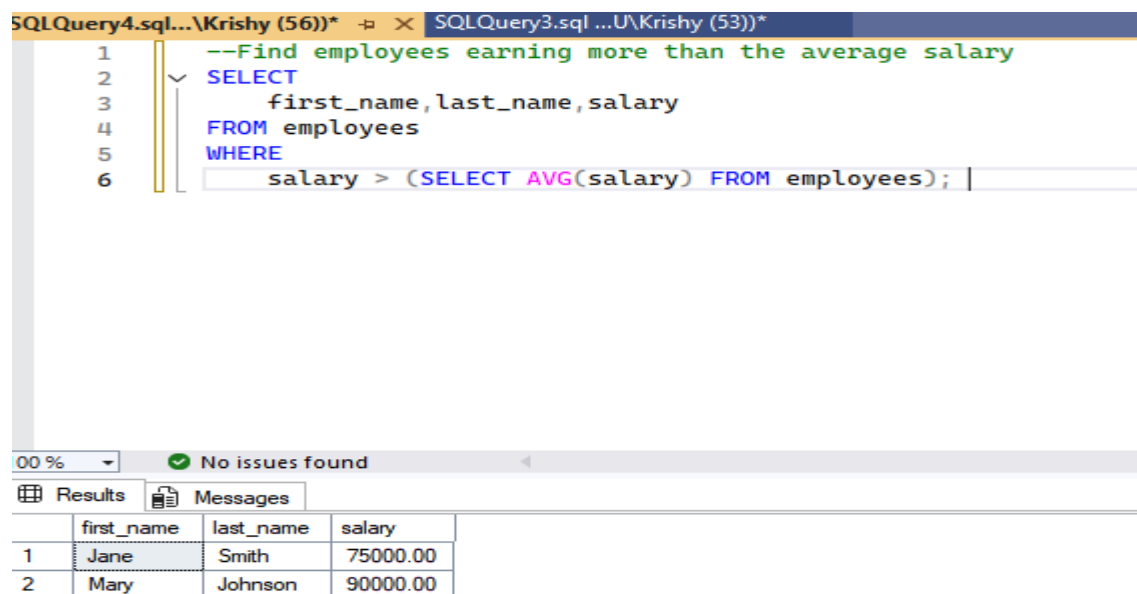
	department_id	department_name
1	10	IT
2	20	Sales
3	30	Finance

SubQuery

A subquery is a SELECT statement that is embedded within another SQL statement (e.g., SELECT, INSERT, UPDATE, DELETE). It is executed first, and its result is then used by the outer query. Subqueries can be used in the SELECT clause, WHERE clause, and FROM clause.

Uses

- Filter results
- Compute values
- Create virtual tables

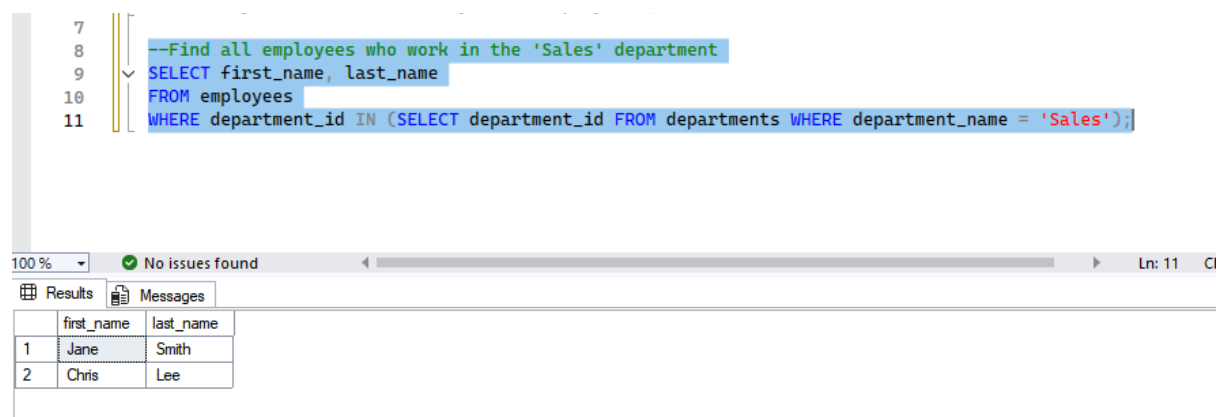


The screenshot shows a SQL query in a text editor. The query is designed to find employees earning more than the average salary. The subquery calculates the average salary from the 'employees' table, and the main query filters for employees whose salary is greater than this average.

```
1  --Find employees earning more than the average salary
2  SELECT
3      first_name, last_name, salary
4  FROM employees
5  WHERE
6      salary > (SELECT AVG(salary) FROM employees);
```

Below the query editor, the 'Results' tab is active, displaying the following data:

	first_name	last_name	salary
1	Jane	Smith	75000.00
2	Mary	Johnson	90000.00



The screenshot shows a SQL query in a text editor. The query is designed to find all employees who work in the 'Sales' department. The subquery identifies the department_id for the 'Sales' department, and the main query filters for employees whose department_id matches this value.

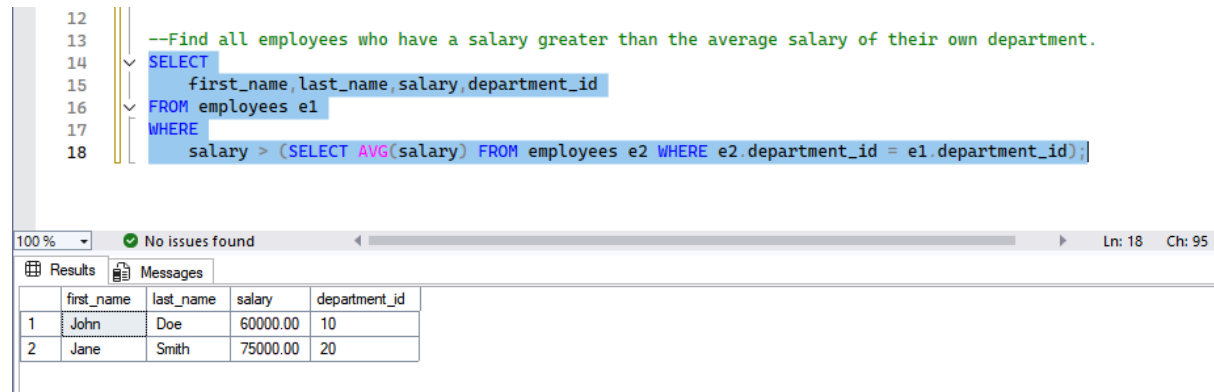
```
7
8  --Find all employees who work in the 'Sales' department
9  SELECT first_name, last_name
10 FROM employees
11 WHERE department_id IN (SELECT department_id FROM departments WHERE department_name = 'Sales');
```

Below the query editor, the 'Results' tab is active, displaying the following data:

	first_name	last_name
1	Jane	Smith
2	Chris	Lee

Correlated subquery

A correlated subquery is a subquery that references a column from the outer query. It is executed once for each row processed by the outer query.



```
12
13 --Find all employees who have a salary greater than the average salary of their own department.
14 SELECT
15     first_name, last_name, salary, department_id
16 FROM employees e1
17 WHERE
18     salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.department_id = e1.department_id);
```

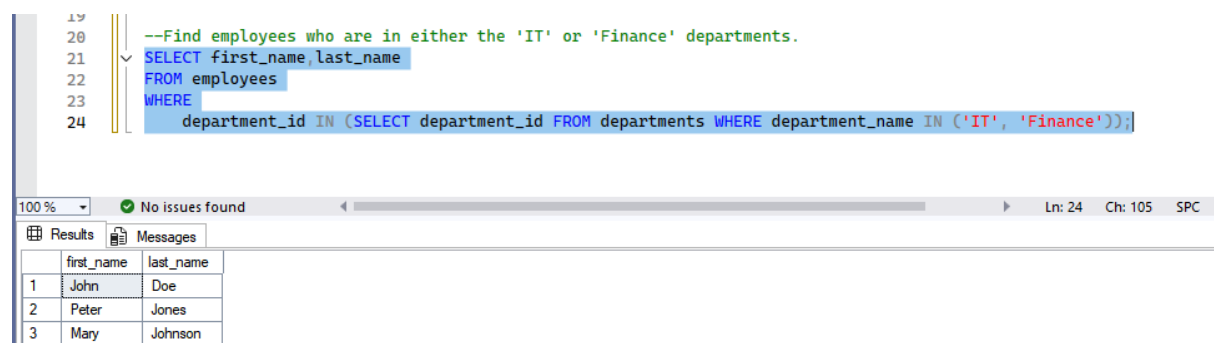
100% No issues found Ln: 18 Ch: 95

	first_name	last_name	salary	department_id
1	John	Doe	60000.00	10
2	Jane	Smith	75000.00	20

For each employee (e1), the inner query calculates the average salary only for that employee's department (e2.department_id = e1.department_id).

Subquery returning multiple rows

Subqueries can return multiple rows. When they do, they must be used with operators that can handle a set of values, such as IN, NOT IN, ANY, or ALL.



```
20 --Find employees who are in either the 'IT' or 'Finance' departments.
21 SELECT first_name, last_name
22 FROM employees
23 WHERE
24     department_id IN (SELECT department_id FROM departments WHERE department_name IN ('IT', 'Finance'));
```

100% No issues found Ln: 24 Ch: 105 SPC

	first_name	last_name
1	John	Doe
2	Peter	Jones
3	Mary	Johnson

Exists

EXISTS is a logical operator that checks for the existence of any rows returned by a subquery. It returns TRUE if the subquery returns at least one row, and FALSE if it returns none. It is often more performant than IN because it stops searching as soon as it finds the first match.

26	--if there exists a department record with a department_name of 'Sales' for each employee
27	SELECT first_name, last_name
28	FROM employees e
29	WHERE EXISTS (
30	SELECT 1
31	FROM departments d
32	WHERE d.department_id = e.department_id AND d.department_name = 'Sales'
33	);

20%	No issues found	Ln: 26	Ch: 90
Results	Messages		
	first_name	last_name	
1	Jane	Smith	
2	Chris	Lee	

This query checks if there exists a department record with a department_name of 'Sales' for each employee. The SELECT 1 is just a convention; the EXISTS operator doesn't care about the returned columns, only whether a row exists.

Performance affected by subqueries?

- **Non-Correlated Subqueries:** The inner query runs once and its result is cached for the outer query. Performance is generally good.
- **Correlated Subqueries:** These can be very slow because the inner query runs for every single row of the outer query. For large datasets, a JOIN is almost always a better, more performant alternative. Modern database optimizers can sometimes rewrite a correlated subquery into a join, but this is not guaranteed.

Scalar subquery

A scalar subquery is a subquery that returns a single value (one row and one column). They can be used anywhere a single value is expected, such as in the SELECT list, WHERE clause, or a SET clause of an UPDATE statement.

```

34
35 --List each employee's salary and compare it to the overall average salary.
36 SELECT
37     first_name, last_name, salary,
38     (SELECT AVG(salary) FROM employees) AS average_salary
39 FROM employees;

```

100 % No issues found

Results Messages

	first_name	last_name	salary	average_salary
1	John	Doe	60000.00	68400.000000
2	Jane	Smith	75000.00	68400.000000
3	Peter	Jones	55000.00	68400.000000
4	Mary	Johnson	90000.00	68400.000000
5	Chris	Lee	62000.00	68400.000000

The subquery in the SELECT list calculates the average salary once and applies it to every row in the result set.

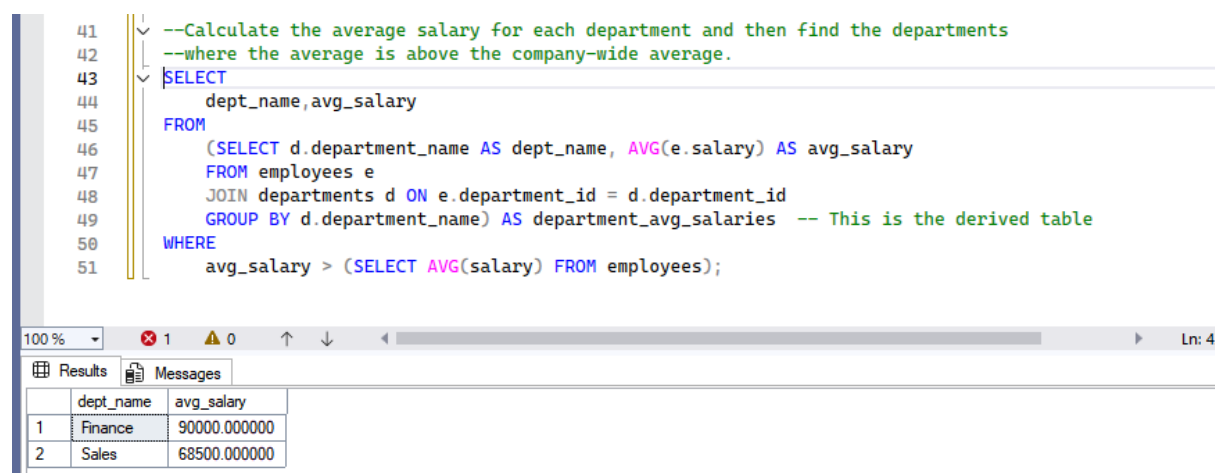
How can be Subqueries used

Subqueries can be used in several places within an SQL statement:

- **SELECT clause:** As a scalar value.
- **FROM clause:** To create a "derived table" (a temporary result set).
- **WHERE clause:** To filter data using operators like IN, NOT IN, =, <, >, EXISTS, NOT EXISTS.
- **HAVING clause:** To filter groups based on an aggregate condition.

Subqueries in FROM clause

Subquery can be used in the FROM clause. The result of the subquery is treated as a temporary, inline table, also known as a **derived table** or **inline view**.



The screenshot shows a SQL query in a text editor with line numbers 41 to 51. The query calculates the average salary for each department and then finds the departments where the average is above the company-wide average. The query uses a subquery in the FROM clause, aliased as 'department_avg_salaries', which is a derived table. The results are displayed in a table below the query.

```
41 --Calculate the average salary for each department and then find the departments
42 --where the average is above the company-wide average.
43 SELECT
44     dept_name, avg_salary
45 FROM
46     (SELECT d.department_name AS dept_name, AVG(e.salary) AS avg_salary
47     FROM employees e
48     JOIN departments d ON e.department_id = d.department_id
49     GROUP BY d.department_name) AS department_avg_salaries -- This is the derived table
50 WHERE
51     avg_salary > (SELECT AVG(salary) FROM employees);
```

The results table shows the following data:

	dept_name	avg_salary
1	Finance	90000.000000
2	Sales	68500.000000

Derived table

A derived table is the name given to a subquery that is used in the FROM clause. It is a temporary, in-memory table created on the fly during the execution of a query. The derived table must have an alias, as shown in the example above (AS department_avg_salaries).

Key Characteristics:

- It's a temporary, non-permanent result set.
- It must have an alias.
- It is useful for breaking down complex problems into smaller, more manageable steps.
- Can sometimes be a good alternative to a CTE (Common Table Expression).

Summary (When to Use What):

- **Scalar Subquery** → return single value (SELECT, WHERE).
- **Correlated Subquery** → when filtering based on outer query row.
- **IN / EXISTS** → when multiple rows possible.
- **FROM (Derived Table)** → treat subquery as a temporary table.

Thank you!